ACADEMIC PROJECT REPORT

Machine-Generated Code Detection and Cyclomatic Complexity Analysis

A SemEval-inspired academic demonstration

ADA Project

Submitted by **Tanush Virmani**

Bachelor of Technology
Computer Science and Engineering

Amity School of Engineering & Technology
Amity University Uttar Pradesh

Batch 2023-27
26 October 2026

Report scope	Recorded outcome
Final reserved holdout	79.66% view accuracy on 1,450 snippets/views
Independent parent-file check	87.22% accuracy across 454 parent files
Reused SemEval public sample	66.30% accuracy on 1,000 examples; exploratory recheck

This report distinguishes the broader academic holdout from the original SemEval public sample. External training data is used in the deployed demo; this is not an official competition submission.

Abstract and overview

This project implements a local web application that estimates whether source code is human-written or machine-generated and independently measures cyclomatic complexity. The initial implementation used CodeBERT as a pretrained encoder, but lacked trained classification weights and a complete evaluation workflow. A runnable training and evaluation pipeline was added, followed by systematic experiments to address poor generalization.

The original 2,000-example CodeBERT experiment obtained 95.8% validation accuracy but only 43.1% accuracy on the 1,000-example public SemEval sample. This gap motivated language/class balancing, broader production-code coverage, function-fragment views and validation-based model selection. The selected deployed model combines lexical TF-IDF classification with structural boosting and randomized-tree classifiers. Its decision threshold is chosen using validation data.

The final reserved holdout contains 1,450 views from 454 previously uninspected parent files. The selected model obtained 79.66% view accuracy and 87.22% parent-file accuracy. The reused SemEval sample obtained 66.30%. These results describe different distributions and are reported separately.

Implemented deliverables

A Gradio interface; validated local checkpoint loading; CPU-efficient and CodeBERT inference; data preparation, training, threshold selection and evaluation scripts; per-example predictions; confusion matrices; provenance metadata; and automated tests.

Report map

Page	Section
3	Problem statement, objectives and ADA relevance
4	Datasets and evaluation design
5	System architecture and workflow
6	Algorithms and implementation
7	Experiments and model selection
8	Final reserved-holdout results
9	SemEval public-sample comparison
10	Verification, limitations and references

1. Problem statement and objectives

Given a source-code snippet and its programming language, the detector assigns one of two labels: **0 - Human-written** or **1 - Machine-generated**. The label is a statistical prediction, not a determination of authorship. Code copied, edited, generated by templates, or jointly authored by people and AI may not fit a strict binary interpretation.

Objectives

1. Make the original project reproducible and prevent inference with randomly initialized classifier weights.
2. Calculate accuracy, precision, recall, F1 and confusion matrices from labeled, held-out examples.
3. Improve detection using broader training coverage and validation-selected models.
4. Expose predictions and cyclomatic-complexity analysis through a local application.
5. Preserve evidence, failed experiments and limitations rather than report only favorable numbers.

Connection to analysis and design of algorithms

The application compares two computational approaches: a transformer encoder and sparse lexical/structural classifiers. The project therefore considers preprocessing, feature extraction, model inference and memory/runtime trade-offs. The CPU ensemble provides a practical alternative when transformer fine-tuning is expensive on the available machine.

Cyclomatic complexity is a separate structural measure of control-flow paths. For a control-flow graph, the standard expression is V(G) = E - N + 2P, where E is the edge count, N the node count and P the number of connected components. The application uses Lizard to identify functions and count language-specific branch constructs. This measure is not the same as asymptotic runtime complexity.

Scope

The academic demo supports the languages exposed by its current classifier and uses a binary authorship task inspired by SemEval Subtask A. It does not implement generator attribution, plagiarism detection or proof of misconduct. HTML can be classified in the broad demo, but cyclomatic-complexity analysis is unavailable for HTML.

2. Data and experimental methodology

Dataset	Use and provenance
SemEval-2026 Task 13, configuration A	Official train/validation data and the 1,000-example public test sample. Training emphasizes Python, C++ and Java.
CodeMirage	Broader academic training coverage, including production-style code and normal/paraphrased AI examples. Original source: HanxiGuo/CodeMirage; license CC BY-NC-ND 4.0.
Final reserved files	454 remaining CodeMirage test parent files, not used in fitting or shared-validation selection.

Preparation and grouping

Empty or missing code/labels are rejected or removed at preparation time. Labels remain exactly 0 and 1. Exact code hashes prevent direct reuse across fitting and evaluation. A conservative normalized fingerprint also removes identifier names, literal values, comments and whitespace to group obvious renamed or reformatted code. Grouping does not establish that all semantically related programs or repository families are independent.

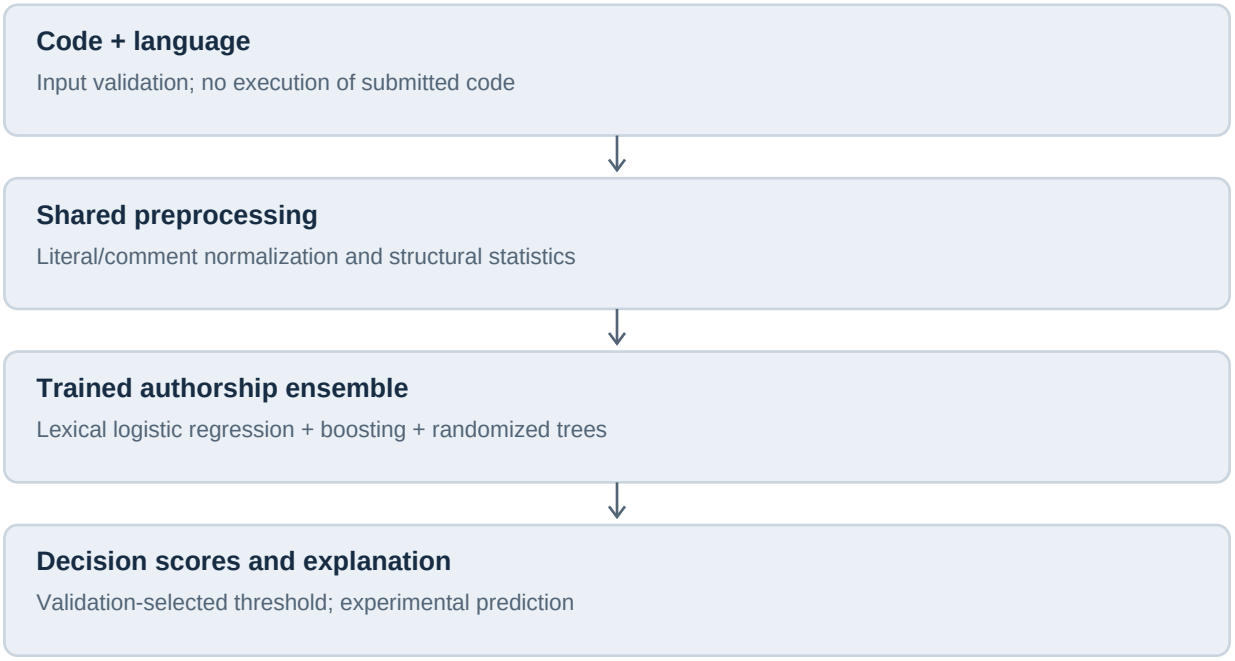
Function views are extracted using Lizard; a deterministic contiguous window is used when suitable functions are unavailable. Each view inherits its parent label. This broadens the input-length distribution, but inherited labels can be noisy when one file contains mixed or generated components.

Separation of roles

Training data fits model parameters. Validation data selects models and thresholds. A shared 1,000-example validation subset compares the completed final candidates. Reserved parent files are evaluated only after this choice is locked. Repeated code patterns matching fitting, tuning or previously inspected views are removed before the final holdout evaluation.

The SemEval public sample was inspected during the original failure analysis. Subsequent results on that sample are development rechecks, not fresh estimates. The broader track uses external training data and is not competition-compliant. Dataset revisions, file hashes and selection records are retained in manifests and training metadata.

3. System architecture



Parallel complexity path

The same input is sent separately to Lizard for function-level complexity, lines of code and token counts. A horizontal chart shows complexity per function. Complexity analysis remains available when classifier weights are missing; the interface reports the detector as unavailable instead of inventing a prediction.

Module responsibilities

Module	Responsibility
app.py	Local Gradio UI, input handling, scores and complexity display
detector.py / model_utils.py	Shared inference, tokenization, checkpoint validation and decision scores
improve_model.py / fragment_views.py	CPU models, language/class balancing and fragment preparation
train_model.py	CodeBERT training, resumable checkpoints and optional layer freezing
evaluate_model.py	Metrics, leakage guards, predictions, parent aggregation and confusion matrix
model_selection.json	Recorded validation-only choice of the deployed model

4. Algorithms and implementation

Selected authorship classifier

The selected model is **lexical_C1.0_ensemble**. Word/token TF-IDF uses 1-2 grams, with an 80,000-feature cap. Character TF-IDF uses 3-5 grams, with a 100,000-feature cap. Comment and literal normalization reduces some superficial differences. Structural statistics include line and identifier lengths, punctuation ratios, indentation and control-flow keyword frequencies.

The lexical branch uses logistic regression with C = 1.0. Two structural branches use histogram gradient boosting and an ExtraTrees ensemble. Their three machine-class scores are averaged. Candidate branches and ensembles are selected using validation macro F1. These feature caps are configuration limits, not claims about the final learned vocabulary size.

Decision threshold

The deployed threshold is t = 0.55, chosen on validation data. For consistent UI decisions, raw machine score p is mapped to p* = p(1-t) / [p(1-t) + (1-p)t]. A displayed machine score above 0.5 is equivalent to a raw score above t. These values are uncalibrated decision scores, not verified probabilities of authorship.

Computational considerations

Operation	Main cost / interpretation
TF-IDF transformation	Character/token scans for fixed n-gram lengths; sparse output
Logistic inference	Sparse dot product, proportional to active feature count
Tree inference	One traversal per tree; depends on tree count and depth
Transformer inference	Per-layer attention has a quadratic dependence on sequence length; feed-forward layers also contribute
Cyclomatic analysis	Static parsing and branch counting; no source code is executed

No controlled latency benchmark was conducted. Layer freezing reduces backpropagation work but does not remove the transformer forward pass. The local CPU was used for the recorded experiments.

5. Experiments and model selection

Experiments were preserved even when their public-sample results were worse. High scores on similar validation data did not guarantee performance across languages, domains or snippet types.

Exploratory experiment	Public SemEval recheck
Original CodeBERT; 2,000 official training examples	43.1%
Balanced 48,000-example CPU ensemble	29.7%
Broader full-file CPU ensemble	49.0%
Fragment-aware ensemble; validation-selected threshold	66.3%

Completed shared-validation comparison

Candidate	Accuracy	Macro F1
Fragment-aware CPU ensemble	79.5%	0.7948
CodeBERT transfer; 8 lower layers frozen	70.1%	0.7006
Imported external detector	55.3%	0.4723

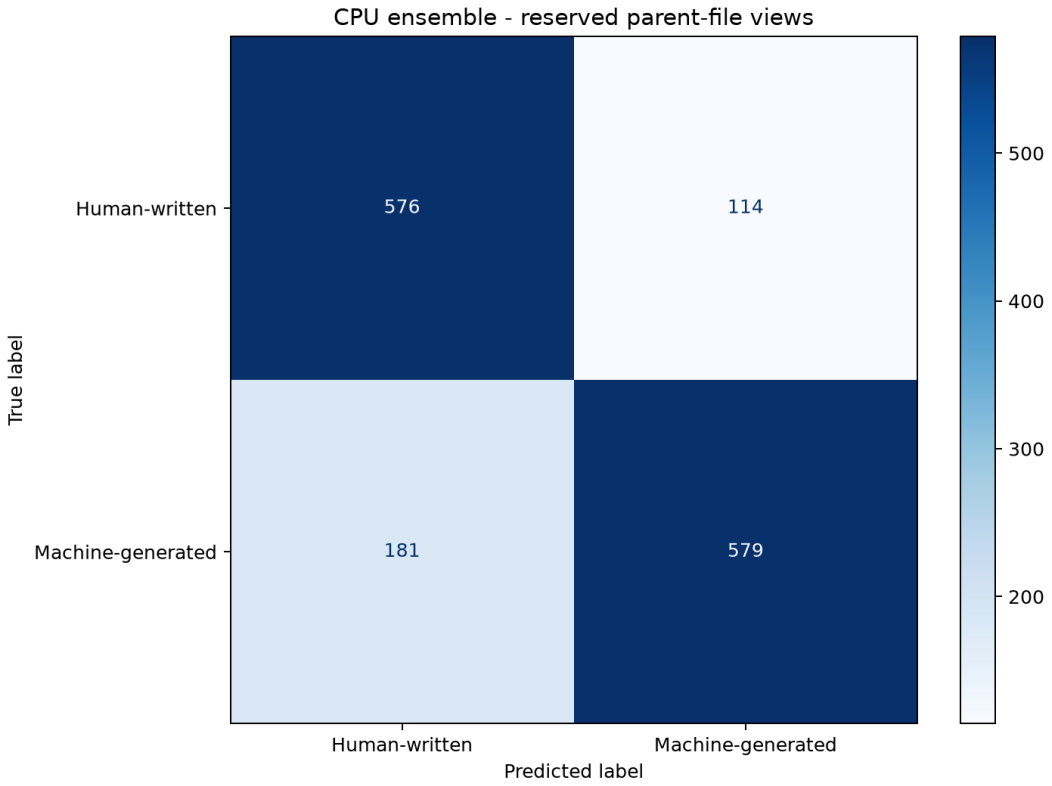
The fragment-aware ensemble was selected using the shared validation macro F1. CodeBERT transfer used 4,000 examples, 128 first-and-last tokens, one epoch, and a 5e-5 learning rate. The external detector was a comparison only; its precise training dataset is not documented by its authors, so its training/evaluation overlap could not be independently checked. It was not deployed.

The final reserved parent files were not used in this candidate selection. The model-selection record includes the validation-file SHA-256 hash and scores for all completed candidates.

6. Final reserved-holdout results

Evaluation used **1,450 snippets/views** from **454 parent files**. The majority-class accuracy baseline is 52.41%. View accuracy is **79.66%**, macro precision 0.7982, macro recall 0.7983 and macro F1 **0.7966**.

Class	Precision	Recall	F1	Support
Human-written	0.7609	0.8348	0.7961	690
Machine-generated	0.8355	0.7618	0.7970	760



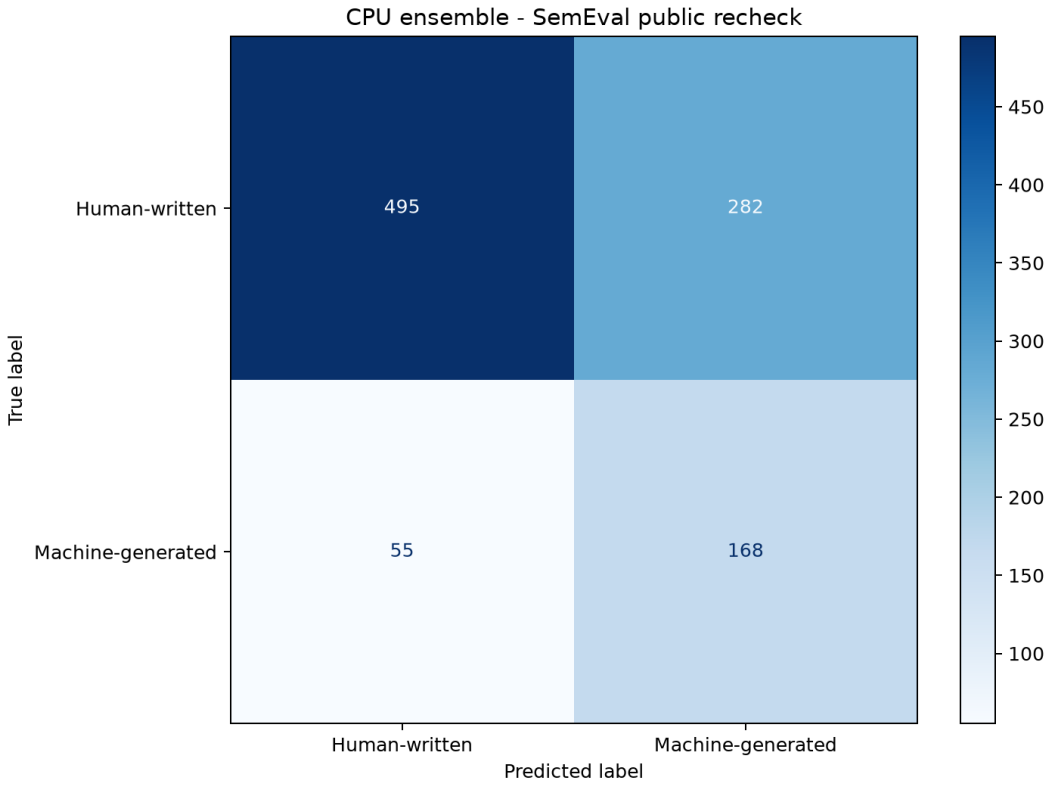
Averaging scores across each parent's views gives **87.22% parent-file accuracy** and 0.8721 macro F1. Views from one parent are correlated; these two levels must not be treated as independent sample counts.

This result measures the specified reserved CodeMirage-based task. It does not establish the same accuracy on all source code, generators, domains or future submissions.

7. Original SemEval sample: separate result

The same 1,000-example public sample used in the original project now gives **66.30%** accuracy, macro precision 0.6367, macro recall 0.6952 and macro F1 0.6227. This is an exploratory recheck after development.

Class	Precision	Recall	F1	Support
Human-written	0.9000	0.6371	0.7460	777
Machine-generated	0.3733	0.7534	0.4993	223



Accuracy improved from 43.1% to 66.3%, but remains below both 75% and the sample's 77.7% majority-class accuracy baseline. The greater-than-75% objective is met on the separately described broader holdout (79.66%); it is **not achieved on this original SemEval sample**. The two benchmarks must not be merged or presented as directly interchangeable results.

8. Verification, limitations and future work

Verification completed

Automated checks cover invalid/fractional/missing labels, both-class requirements, missing checkpoints, exact training/evaluation overlap, normalization, finite structural features, deterministic fragment extraction, threshold mapping, and first-and-last token handling. Saved predictions are checked against accuracy, macro F1 and confusion matrices. The local application was opened and tested through its browser interface.

Limitations and next steps

Source-level labels can be noisy; function views may inherit imperfect labels. Normalized fingerprints cannot detect all semantic clones or shared repositories. Confidence is uncalibrated, and no detector output should be treated as proof of misconduct. Larger and better-grouped datasets, generator/repository-held-out validation, stronger code encoders, and carefully controlled GPU experiments are appropriate next steps. Thresholds must not be tuned on final test labels simply to exceed a target accuracy.

Reproduction and local use

README.md and IMPROVEMENTS.md describe the environment and experiment commands. Run app.py from the project environment to use http://127.0.0.1:7860. active_model.json identifies the selected local checkpoint. The repository retains the original CodeBERT model and results as separate local artifacts. Raw datasets, caches and large local weights are ignored by Git.

References and software sources

[1] SemEval-2026 Task 13 organizers. Task definition and restrictions. https://github.com/mbzuai-nlp/SemEval-2026-Task13

[2] DaniilOr/SemEval-2026-Task13. Official dataset. https://huggingface.co/datasets/DaniilOr/SemEval-2026-Task13

[3] HanxiGuo/CodeMirage. Dataset card and provenance. https://huggingface.co/datasets/HanxiGuo/CodeMirage

[4] Microsoft CodeBERT. Pretrained code encoder. https://github.com/microsoft/CodeBERT

[5] Lizard static complexity analyzer. https://github.com/terryyin/lizard

[6] scikit-learn. Feature extraction and classifier implementations. https://scikit-learn.org/stable/

[7] azherali/CodeGenDetect-CodeBert. External comparison model card. https://huggingface.co/azherali/CodeGenDetect-CodeBert